

Memory Hierarchy

DSC 102: Systems for Scalable Analytics

Haojian Jin

UC San Diego

Important Information About Your Spending Ability >



American Express <AmericanExpress@welcome.aexp.com>
to me ▾

Thu, Aug 3, 2017, 6:41AM ☆ ↶

Important information regarding your recent activity

HAOJIAN JIN

Account Ending:
02005

View Account

Recent Activity

Transaction Declined

Dear HAOJIAN JIN,

We were unable to authorize the below listed purchase on your American Express Card referenced above.

Reason: The transaction would have caused your balance to exceed your credit limit.

Merchant	Amazon Web Services
Transaction Amount	\$43,845.21
Balance* (including transaction above)	\$44,140.00
Credit Limit	\$15,000.00

To use your Card without interruption please spend within your credit limit. You may make a **payment** to bring down any existing balance.

Make a Payment

or

Call us at 1-800-472-9297

As a reminder, the [Check Spending Power](#) tool is available to verify if a future charge will be approved.

copyij vs. copyji: Copy a 2048×2048 integer array

```
void copyij(long src[2048][2048], long dst[2048][2048]) {  
    for (long i = 0; i < 2048; i++)  
        for (long j = 0; j < 2048; j++)  
            dst[i][j] = src[i][j];  
}  
  
void copyji(long src[2048][2048], long dst[2048][2048]) {  
    for (long j = 0; j < 2048; j++)  
        for (long i = 0; i < 2048; i++)  
            dst[i][j] = src[i][j];  
}
```

Same number of copies. Same result. Which one is faster?

copyij vs. copyji: Results

copyij : **4.3 ms**

copyji : **81.8 ms**

- ~**20×** slower, just by swapping two loops
- Nothing in the code "looks" more expensive

Try it yourself: `copy_demo.c`

```
cc -O0 -o copy_demo copy_demo.c && ./copy_demo    # no optimization
cc -O2 -o copy_demo copy_demo.c && ./copy_demo    # optimized
```

2048×2048 `long`, Apple M4 Pro, clang 17

Build	copyij	copyji	Gap
-O0 (every variable lives in memory)	4.7 ms	29.5 ms	6×
-O2 (registers + SIMD vectorization)	0.6 ms	38.9 ms	62×

- -O2 copies copyij's row **64 bytes per step**; copyji's column can't be vectorized
- The compiler makes the good order faster, but **cannot rescue the bad order**

Try it yourself: `copy_demo.py`

Python + NumPy, 2048×2048 int64, Apple M4 Pro

Version	<code>copyij</code>	<code>copyji</code>	Gap
Pure Python loops	234 ms	389 ms	1.7×
NumPy, row-major (C order)	0.9 ms	49.3 ms	55×
NumPy, column-major (Fortran order)	45.9 ms	0.8 ms	flipped

Questions for this lecture

- How to measure the speed of a program?
- Why is C faster than Python? Why is NumPy faster than Pure Python? Why is Numpy slightly faster than unoptimized C?
- Why is copyij faster than copyji?
- How can we make copyij even faster?

By the end of this lecture, you need to be crystal clear on the answer to each of these questions.

Q1: How to measure the speed of a program?

Speed = time to finish a workload

- **Wall-clock time** (`real`): what the user waits for
- **CPU time** (`user` + `sys`): time the CPU actually spent on your program
- **Latency**: time for one task · **Throughput**: tasks per second

```
$ time ./copy_demo  
real 0.40    user 0.22    sys 0.00
```

- But the copy itself took **0.7 ms**: the rest is setup, checks, and repeats
- **Measure the part you care about**

Measuring in practice

```
t0 = time.perf_counter()
copyij(src, dst)
elapsed = time.perf_counter() - t0
```

- Run several times; report the **min or median** (the first run is "cold")
- Keep the machine quiet (close the browser, no other jobs)
- Always report the **hardware**: same code, different machine → different number

The software view: count cycles

Time = Instructions × Cycles per instruction × Seconds per cycle

- **Instructions:** the algorithm, the language, the compiler
- **Cycles per instruction:** how well the CPU executes them
- **Seconds per cycle:** the clock rate (M4 Pro $\approx$ 4.5 GHz $\rightarrow$ **0.22 ns** per cycle)

copyij and copyji, counted in cycles

$2048 \times 2048 = 4.2 \text{ M copies}$, clock $\approx 4.5 \text{ GHz}$

Version	Time	Cycles per copy
copyij -00	4.7 ms	~ 5
copyij -02	0.6 ms	~ 0.6
copyji -02	38.9 ms	~ 42

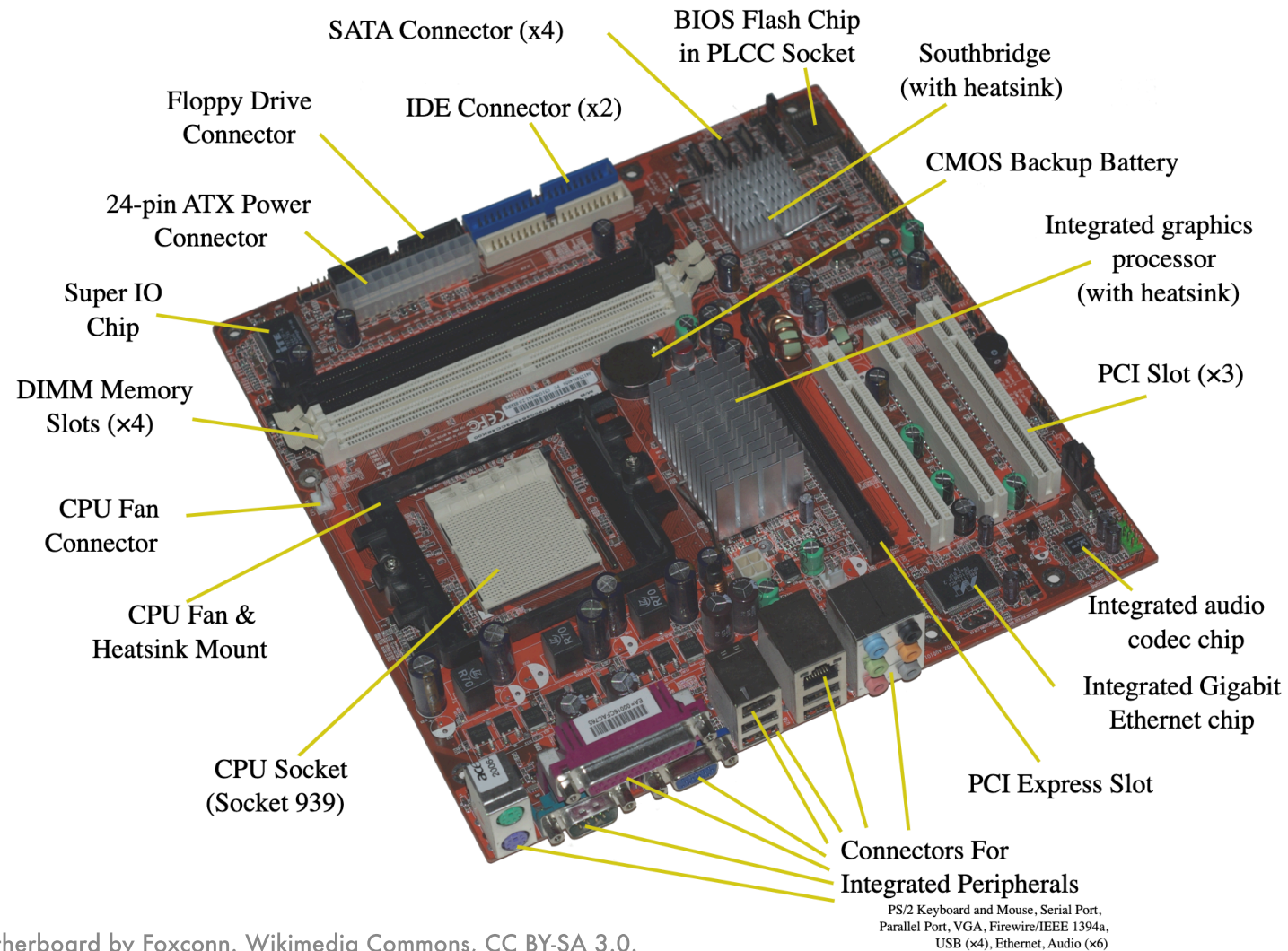
- < 1 cycle per copy: the hardware does **several copies at once**
- 42 cycles per copy: the CPU is mostly **waiting for memory**
- Same instructions, $70\times$ difference $\rightarrow$ **the hardware matters**

The hardware view: what limits speed?

- **Compute**: operations per second = clock × cores × operations per cycle
- **Parallelism**: can the work be split across cores, SIMD lanes, GPUs, machines?
- **Data movement (I/O)**: how fast data arrives from memory, disk, network
- A program runs at the speed of its **bottleneck**
 - compute-bound vs. memory-bound vs. I/O-bound

To reason about this, we need to know what is inside a computer.

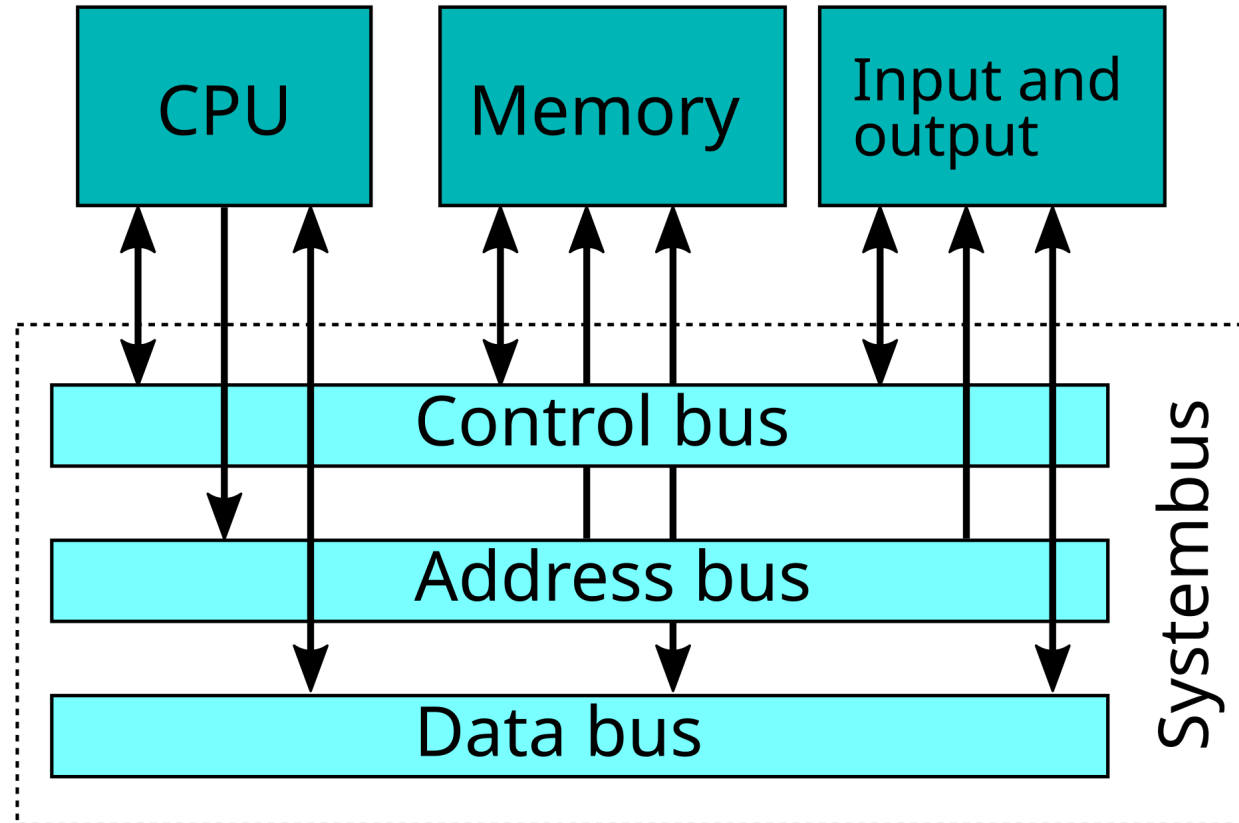
What's inside a computer?



Key parts of computer hardware

- **Processor** (CPU, GPU, etc.)
 - Orchestrates and executes *instructions* to manipulate *data*, as specified by a *program*
- **Main memory** (DRAM, Dynamic Random Access Memory)
 - Stores *data* and *programs* for very fast retrieval; byte-level *addressing*
- **Disk** (secondary / persistent storage)
 - Like memory, but *persistent*, *slower*, and higher capacity per dollar
- **Network interface controller** (NIC)
 - Sends and receives data over a network of computers
- **Buses**: the wires that connect them

How the components connect



Inside the processor

- **ALU**: does the arithmetic; **control unit**: decides what runs next
- **Registers**: a few hundred bytes, the fastest storage
- **Caches** (L1/L2/L3): small, fast copies of recently used memory
- **Cores**: several independent processors on one chip
- **SIMD**: one instruction on several values at once
 - copyij -02 moves **64 bytes** per step

My laptop: Apple M4 Pro, 14 cores (10 performance + 4 efficiency), 48 GB memory

Every component has a speed

Component	Latency (one access)	Bandwidth	Typical capacity
Register	~0.3 ns (1 cycle)	—	~KB
L1 / L2 cache	1–5 ns	~1 TB/s	KB – MB
DRAM	~100 ns	100–500 GB/s	16 GB – 2 TB
NVMe SSD	~100 μ s	3–14 GB/s	1 – 30 TB
HDD	~10 ms	~0.2 GB/s	10 – 30 TB
Network (same datacenter)	~0.5 ms	1–50 GB/s	—

- One DRAM access $\approx$ **400 CPU cycles**
- One HDD access $\approx$ **100,000 DRAM accesses**

Keeping the processor busy

- A CPU can do **~100 G** operations per second
- If each operation reads one 8-byte number → needs **800 GB/s** of data
- What can deliver it?
 - DRAM: **~100–500 GB/s**
 - SSD: **~7 GB/s**
 - HDD: **~0.2 GB/s**
- → Most of the time, the processor is **waiting for data**

CPU vs. GPU

CPU vs. GPU: a demonstration

Video player
configuration error

Error 153

Watch video on YouTube

Learn more

Error 153

Watch: how does each one paint the picture? · youtube.com/watch?v=gJ84mx3HbY8

Different design goals

CPU

- **Few, powerful cores** (8 – 192)
- Large caches, complex control
- Optimized for **latency**: finish one task fast
- Good at: OS, databases, pandas, branchy code

GPU

- **Thousands of simple cores**
- Small caches, huge memory bandwidth
- Optimized for **throughput**: many identical tasks
- Good at: matrix multiply, deep learning, graphics

In the video: the CPU robot paints one dot at a time, fast; the GPU fires all the dots at once.

CPU vs. GPU in numbers

	Laptop CPU (Apple M4 Pro)	Server CPU (AMD EPYC 9965)	GPU (NVIDIA H100 SXM)
Cores	14	192	16,896 CUDA cores
Memory bandwidth	273 GB/s	~600 GB/s	3,350 GB/s
Memory size	up to 64 GB	up to 6 TB	80 GB
Power	tens of W	500 W	700 W
Price	part of a ~\$2,000 laptop	~\$15,000	~\$25,000 – 30,000

- A GPU trades memory **size** for memory **speed**

When does a GPU help?

- **Yes:** the same operation on lots of data (matrix multiply, image filters)
- **No:** branchy, sequential, or small tasks
- **Hidden cost:** data must first move CPU memory → GPU memory over PCIe (~64 GB/s)

Would copyij be faster on a GPU?

- Copy inside GPU memory: $64 \text{ MB} \div 3,350 \text{ GB/s} \approx \mathbf{0.02 \text{ ms}}$
- Moving the 32 MB array to the GPU first: $32 \text{ MB} \div 64 \text{ GB/s} \approx \mathbf{0.5 \text{ ms}}$, about the same as the CPU

Your laptop vs. a server

Laptop vs. server

	MacBook Pro (M4 Pro)	Cloud GPU server (AWS p5.48xlarge)
CPU	14 cores	192 vCPUs (AMD EPYC)
Memory	48 GB	2 TB
Storage	1 TB SSD	8 × 3.84 TB NVMe SSD
GPU	integrated	8 × H100 (640 GB GPU memory)
Network	Wi-Fi, ~1 Gb/s	3,200 Gb/s
Power	~100 W	~10 kW

- Same kinds of parts, just **much more of each**

Why are they different?

Laptop

- **One user**, interactive
- Battery life, heat, weight
- Idle most of the day
- ~\$2,000

Server

- **Many users and jobs**, 24/7 → throughput
- Reliability: ECC memory, redundant power, hot-swap disks
- Density and cooling: many servers per rack
- Managed remotely; scaled out over a fast network

How much does it cost?

GPU server: 8× NVIDIA H200 NVL

Description	Quantity	Unit Price	Extended Amount
Exxact TensorEX TS4-190785987 - 4U Dual AMD EPYC 9005 8-GPU MGX Server - 8x NVIDIA H200 NVL	1	\$298,357.60	\$298,357.60
TS4-190785987 - 4U Dual AMD EPYC 9005, 8-GPU MGX Server  - CPU Support: Dual Socket AMD EPYC 9005 (Air cool with up 500W TDP) - Memory: 24x DIMM Slots (12x Channels Per CPU) Up To 6400MT/s (1DPC) - Rear PCIe: 8x PCIe 5.0 x16 FH/FL Double Wide GPU Slots (by PLX Switch), 4x PCIe 5.0 x16 FH/FL Single Width Slots (by PLX Switch), 1x PCIe 5.0 x16 FH/FL Single Wide Slot (from CPU0) - Storage: 8x Front Hot-swap U.2 PCIe 5.0 x4 NVMe (by PLX switch), 2x M.2 PCIe 3.0 x2 2280/22110 - Networking: 2x 10GBase-T (RJ-45) Intel X710-AT2 + Dedicated GbE Management/IPMI Port w/ mini Display port - Power Supply: 4x (3+1) 3200W 80 PLUS Titanium PSU, 100-127Vac @ 1200w, 200-219Vac @ 3000W, 220-240Vac @ 3200w, C20 Inlets - Server Depth: 31.4in / 800mm - Rackmount Rails: Included - TPM Module: On-Board TPM 2.0 Included - Power Cables: Available By Request	1		
AMD EPYC Turin 9555 - 64-Core, 128-Threads, 3.2GHz Base, 256MB Cache - SP5 LGA-6096 - 360W	2		
64GB DDR5 6400MT/s ECC REG Memory Module 1.536TB Total System Memory	24		
2TB M.2 NVMe PCIe 4.0 SSD	2		
7.68TB U.2/U.3 2.5in NVMe PCIe 5.0 Enterprise SSD	2		
--- OS MDADM RAID1: 2x 2TB M.2 NVMe PCIe 4.0 SSD	1		
--- DATA [1-2]: 1x 2TB M.2 NVMe PCIe 5.0 SSD	1		
NVIDIA H200 Hopper NVL 141GB Graphics Card - PCIe 5.0 - 600W <i>Includes 5yr NVIDIA AI Enterprise</i>	8		
NVIDIA NVLink Bridge - 4-way Passive for H200 Hopper NVL PCIe	2		
Ubuntu Linux 24.04 LTS	1		
Broadcom BCM57414 25GbE SFP28 Dual Port Ethernet PCIe Adapter (No Transceiver Modules Included)	1		
ExxactCare 3-Year Parts & Labor	1		
----- ACCESSORIES -----			
6-foot C19 to C20 Power Cable	4		
EDU Discount Applied	1		

GPU server: 8× NVIDIA RTX PRO 6000

Description	Quantity	Unit Price	Extended Amount
Exxact TensorEX TS4-190785987 - 4U Dual AMD EPYC 9005 8-GPU MGX Server - 8x RTX Pro 6000 Blackwell Server Edition	1	\$194,785.61	\$194,785.61
<u>TS4-190785987 - 4U Dual AMD EPYC 9005, 8-GPU MGX Server</u>  - CPU Support: Dual Socket AMD EPYC 9005 (Air cool with up 500W TDP) - Memory: 24x DIMM Slots (12x Channels Per CPU) Up To 6400MT/s (1DPC) - Rear PCIe: 8x PCIe 5.0 x16 FH/FL Double Wide GPU Slots (by PLX Switch), 4x PCIe 5.0 x16 FH/FL Single Width Slots (by PLX Switch), 1x PCIe 5.0 x16 FH/FL Single Wide Slot (from CPU0) - Storage: 8x Front Hot-swap U.2 PCIe 5.0 x4 NVMe (by PLX switch), 2x M.2 PCIe 3.0 x2 2280/22110 - Networking: 2x 10GBase-T (RJ-45) Intel X710-AT2 + Dedicated GbE Management/IPMI Port w/ mini Display port - Power Supply: 4x (3+1) 3200W 80 PLUS Titanium PSU, 100-127Vac @ 1200w, 200-219Vac @ 3000W, 220-240Vac @ 3200w, C20 Inlets - Server Depth: 31.4in / 800mm - Rackmount Rails: Included - TPM Module: On-Board TPM 2.0 Included - Power Cables: Available By Request	1		
AMD EPYC Turin 9555 - 64-Core, 128-Threads, 3.2GHz Base, 256MB Cache - SP5 LGA-6096 - 360W	2		
64GB DDR5 6400MT/s ECC REG Memory Module 1.536TB Total System Memory	24		
2TB M.2 NVMe PCIe 4.0 SSD	2		
7.68TB U.2/U.3 2.5in NVMe PCIe 5.0 Enterprise SSD	2		
--- OS MDADM RAID1: 2x 2TB M.2 NVMe PCIe 4.0 SSD	1		
--- DATA [1-2]: 1x 2TB M.2 NVMe PCIe 5.0 SSD	1		
NVIDIA RTX PRO 6000 Blackwell Server Edition - 96 GB GDDR7, PCIe 5.0 x16, 16-pin, Passive Design, 600W Max TDP	8		
Broadcom BCM57414 25GbE SFP28 Dual Port Ethernet PCIe Adapter (No Transceiver Modules Included)	1		
Ubuntu Linux 24.04 LTS	1		
ExxactCare 3-Year Parts & Labor	1		
----- ACCESSORIES -----			
6-foot C19 to C20 Power Cable	4		
EDU Discount Applied	1		

H200 NVL vs. RTX PRO 6000

Comparing **one GPU**. RTX here means **Blackwell Server Edition**.

Specification	H200 NVL	RTX PRO 6000
Architecture	Hopper	Blackwell
GPU memory	141 GB HBM3e	96 GB GDDR7
Memory bandwidth	4.8 TB/s	1.597 TB/s
Peak FP32 compute	60 TFLOPS	120 TFLOPS
Maximum GPU power	600 W	600 W

- H200: **1.47× the memory capacity, 3× the memory bandwidth**
- RTX: **2× the peak FP32 compute** (32-bit floating point)

Performance depends on the workload

Scan **48 GB** already in **GPU memory**, reading every byte once:

	H200 NVL	RTX PRO 6000
Peak memory bandwidth	4,800 GB/s	1,597 GB/s
Ideal time = 48 GB ÷ bandwidth	10 ms	≈30 ms

- These are **theoretical lower bounds**. Real execution adds overhead.
- A compute-bound FP32 kernel may favor RTX's higher compute rate
- RTX also has **ray-tracing cores** and **native FP4 Tensor Cores**

The bottleneck decides which specification matters.

One job across several GPUs

H200 NVL

- **NVLink** connects groups of 2 or 4 GPUs
- The quote includes **two 4-GPU bridges**
- Each 4-GPU group has **564 GB** of total GPU memory

RTX PRO 6000

- GPU communication uses **PCIe Gen 5**, without NVLink
- Eight GPUs have **768 GB** of total GPU memory
- Independent jobs can use separate GPUs with little communication

Splitting one job across GPUs adds **communication cost**. Each GPU still has its own memory, so the program must distribute the data.

What does the extra \$104,000 buy?

Full server quote	8× H200 NVL	8× RTX PRO 6000
Quoted price	\$298,357.60	\$194,785.61
Total GPU memory	1,128 GB	768 GB

- H200 costs **\$103,571.99 more**, a **53% premium**
- It provides more GPU memory, higher memory bandwidth, and NVLink
- Better throughput per purchase dollar requires **over 1.53× the measured throughput** on your workload

Which resource limits the job you actually want to run?

Prices: the two Exxact server quotes shown above, including CPUs, RAM, storage, and support. H200 also includes five years of NVIDIA AI Enterprise. Purchase-price comparison excludes operating costs.

Training vs. serving: which GPU fits?

Workload	What does the GPU need to do?	Starting choice
Large training jobs	Store activations, gradients, and optimizer state. Synchronize gradients across GPUs.	H200 NVL : more memory and NVLink
Cost-conscious serving	Run forward passes with fixed weights. Model + KV cache + runtime fit in 96 GB.	RTX PRO 6000 : lower-cost replicas and FP4 support
Low-latency LLM serving	Generate tokens quickly, often limited by memory bandwidth at small batches.	H200 NVL : faster memory

Training updates the model. Serving runs an already trained model for users.

Both can train and serve. RTX also suits smaller training and fine-tuning jobs.

Hardware prices (approx., Sept 2026)

Component	Example price	Per GB
Desktop CPU	\$300 – 600	
Server CPU (192-core EPYC)	~\$15,000	
DRAM (DDR5)	32 GB ≈ \$450	~\$15
NVMe SSD	1 TB ≈ \$100	~\$0.10
HDD	20 TB ≈ \$400	~\$0.02
Consumer GPU (RTX 5090)	\$2,000+	
Datacenter GPU (H100)	\$25,000 – 30,000	

- Faster storage costs more per byte: DRAM $\approx 150\times$ SSD $\approx 750\times$ HDD
- DRAM prices rose several-fold since 2025: AI datacenters buy most of it

Cloud: rent instead of buy

Resource (AWS, us-east-1, on-demand)	Price
General VM: 2 vCPUs, 8 GB (m7i.large)	~\$0.10 / hour
8 × H100 server (p5.48xlarge)	~\$55 / hour ($\approx$ \$6.88 per GPU-hour)
Object storage (S3)	~\$0.023 / GB-month
SSD block storage (EBS gp3)	~\$0.08 / GB-month
Data out to the internet	~\$0.09 / GB

- Smaller GPU clouds rent H100s for ~\$2 – 3 / GPU-hour

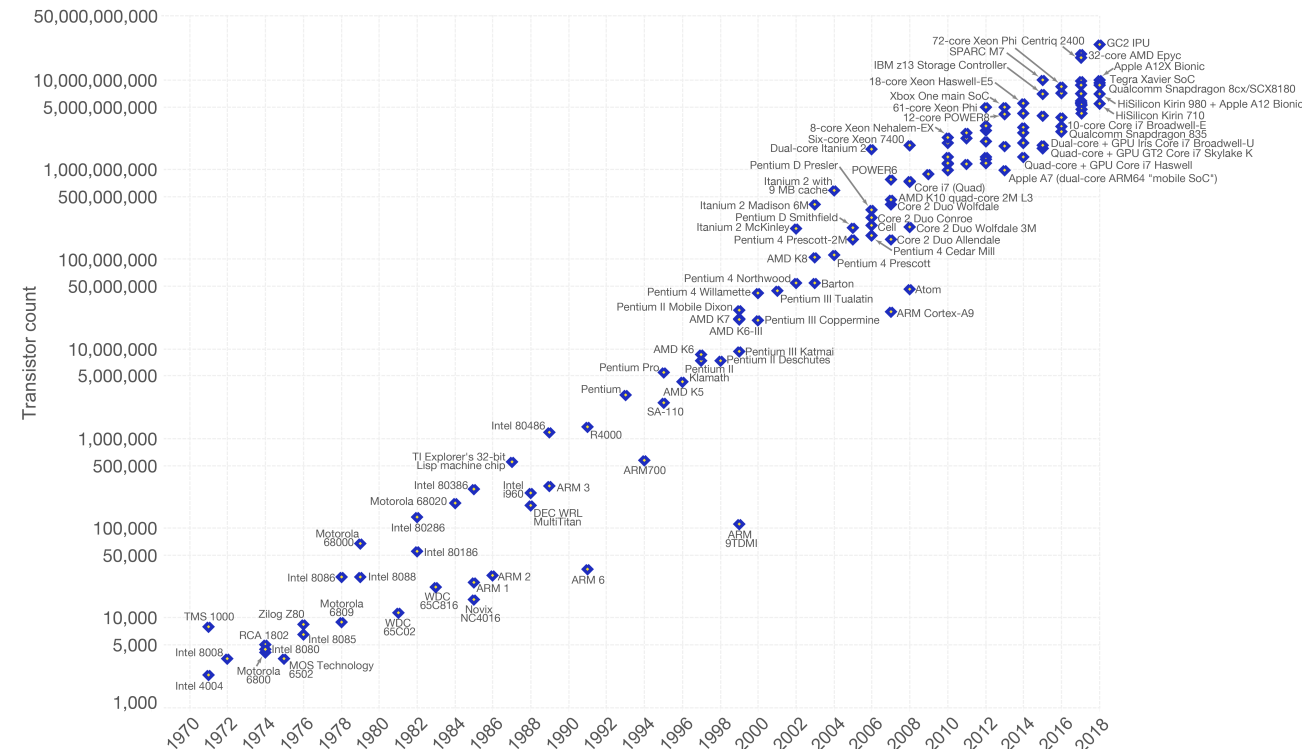
Buy or rent? Quick math

- One H100: ~ **\$27,000** to buy vs. **\$6.88 / hour** on AWS
 - Break-even $\approx 3,900$ hours $\approx$ **5 months of 24/7 use**
 - Buying also means a server, power, cooling, networking, and people
- 1 TB for a year: S3 $\approx$ **\$276**; a 1 TB SSD $\approx$ **\$100**
- Downloading 1 TB out of AWS $\approx$ **\$90**
- Rule of thumb: cloud wins for **bursty** work; owning wins for **steady 24/7** load

Moore's Law

Moore's Law – The number of transistors on integrated circuit chips (1971-2018)

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are linked to Moore's law.



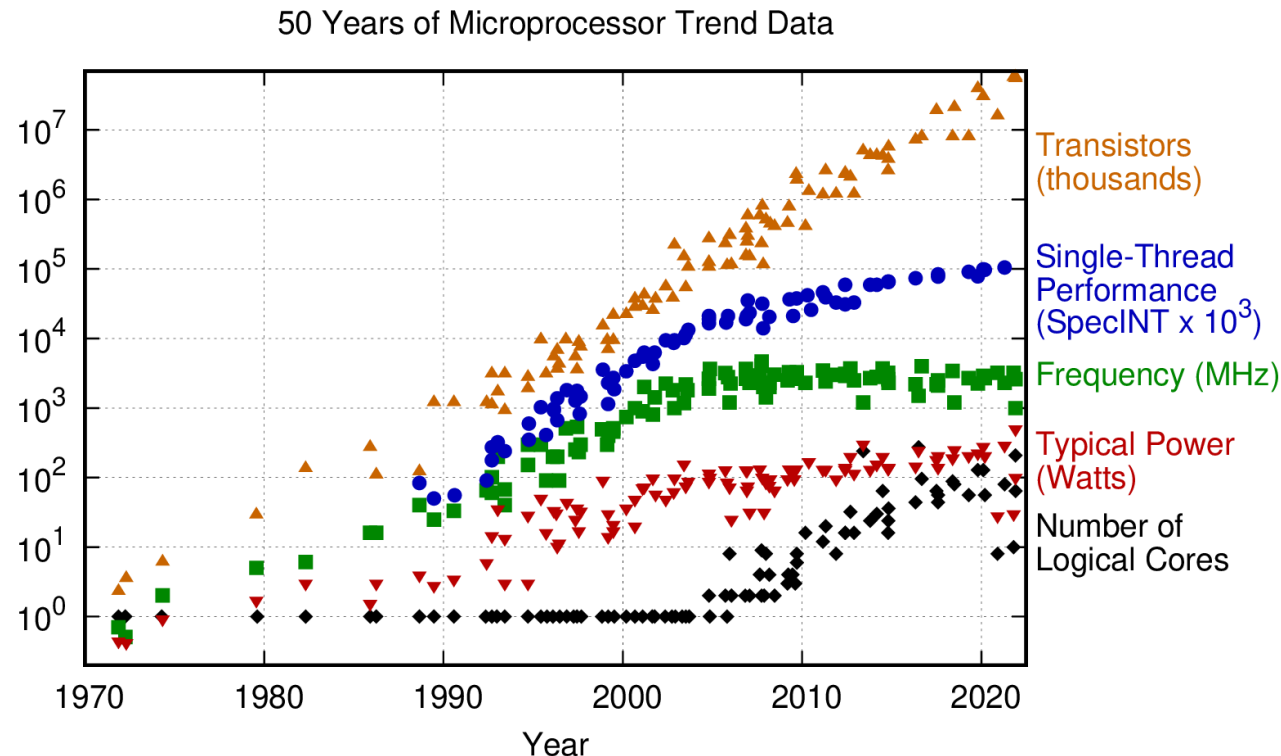
Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at [OurWorldinData.org](https://ourworldindata.org). There you find more visualizations and research on this topic.

Licensed under [CC-BY-SA](#) by the author Max Roser.

- Roughly **$2\times$ as many transistors per chip every two years**
- **Logarithmic y-axis:** equal steps represent equal ratios
- At that rate, ten years gives **$2^5 = 32\times$ the transistors**

Program speed still depends on **architecture and software.**

Transistors, clock speed, and parallelism

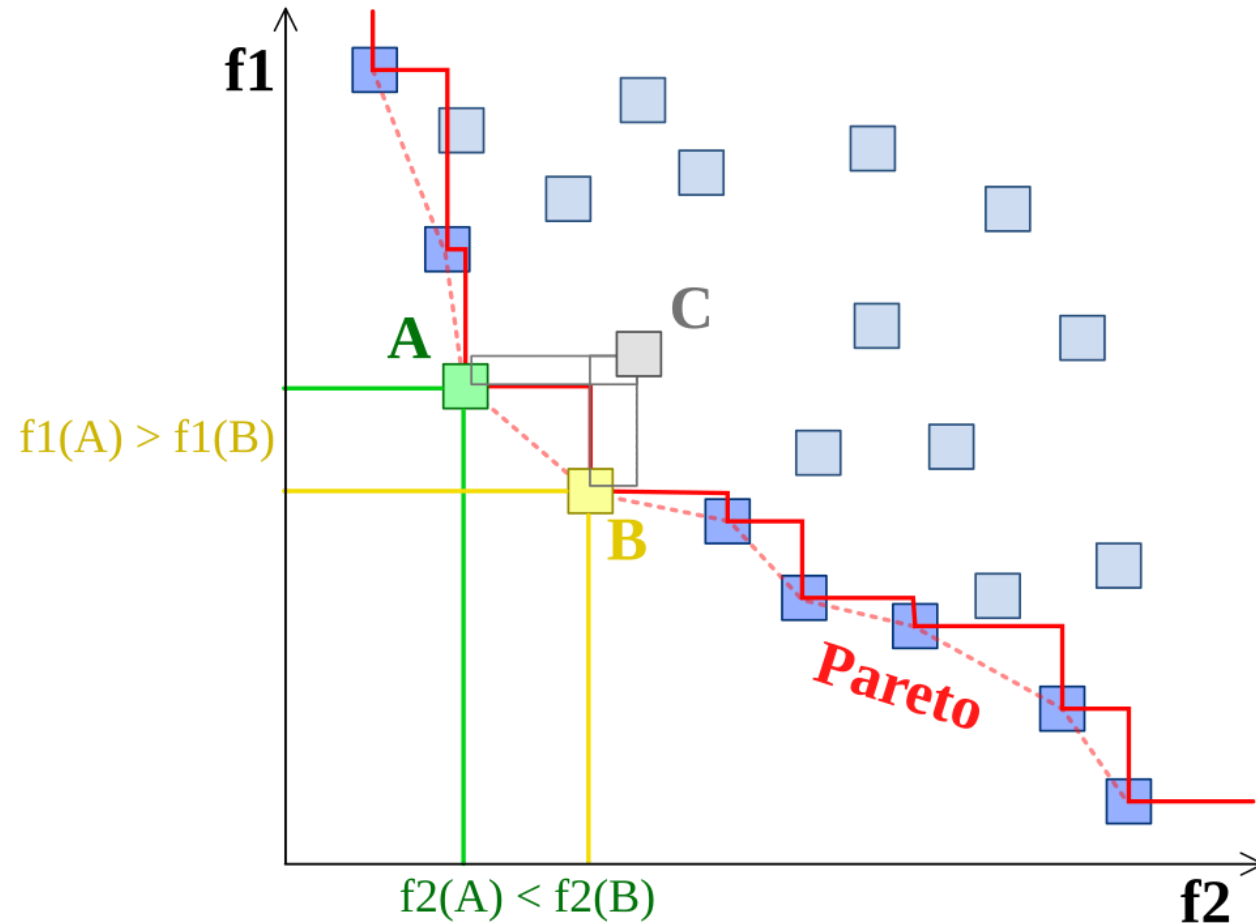


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

- Transistor counts kept rising as **clock growth slowed** in the mid-2000s
- More transistors can fund **more cores and larger caches**
- Software gains speed through **parallel work** and **data locality**

Why can copyji still spend most of its time waiting for memory?

Pareto dominance and the frontier



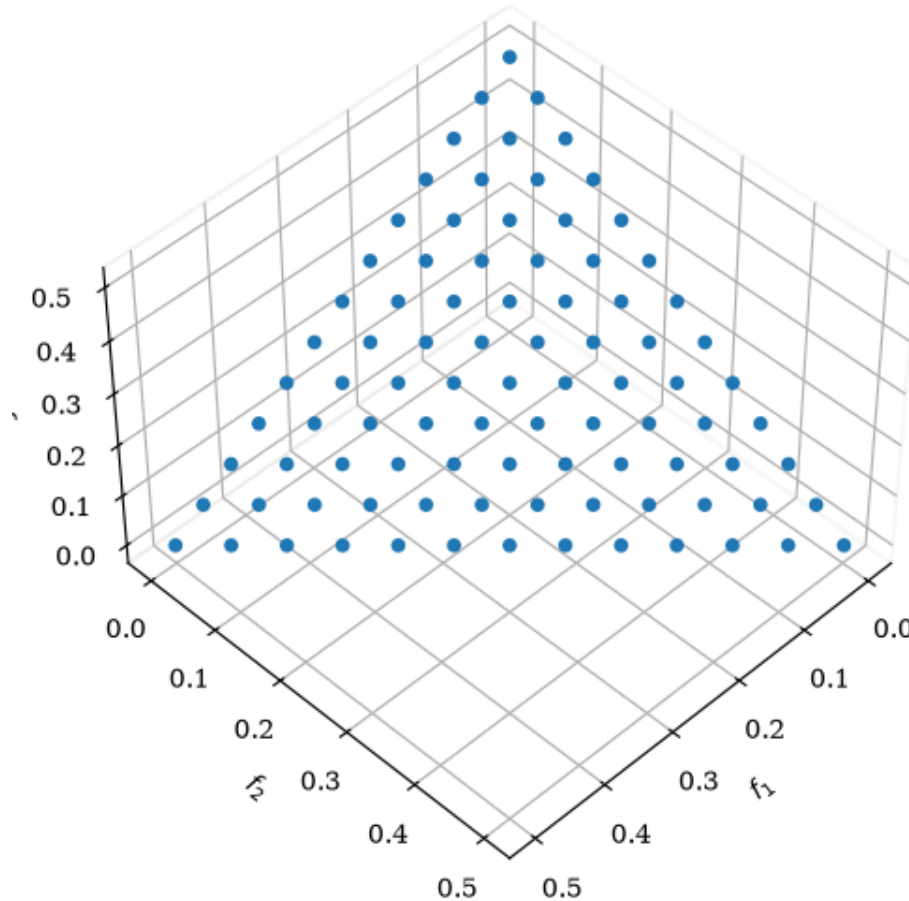
Lower f_1 and f_2 are better. Think **latency** and **cost**.

- **A and B both dominate C**
- **A vs. B** trades off the two objectives

Dominates: no worse on any objective, better on at least one.

The **Pareto frontier** contains all non-dominated points.

Three objectives: a Pareto surface



For hardware, consider **latency, cost, and power**.

- f_3 is the vertical axis.
Minimize all three objectives.
- Non-dominated points can form a **surface**
- Improving one objective requires giving up something on another

Real hardware offers a **discrete set of choices**.

A hardware choice on the Pareto surface

Illustrative numbers, same workload and output quality. Lower is better.

System	Purchase cost	Latency	Power
A	\$10,000	12 ms	200 W
B	\$15,000	8 ms	300 W
C	\$20,000	6 ms	450 W
D	\$18,000	10 ms	350 W

- **B dominates D**: cheaper, faster, and lower power
- **A, B, and C** are non-dominated within this set
- With **budget $\leq$ \$15,000** and **latency $\leq$ 10 ms**, choose **B**

New technology can move the frontier. Our requirements choose a point on it.

Pareto surfaces for language models

Optimize **task quality** ↑, **response latency** ↓, and **cost per task** ↓.

Models	Trade-off they target
Claude / GPT-5.x	Strong reasoning and coding; trade thinking time and cost for quality
Gemini Flash-Lite	Low cost and latency for high-volume, simpler tasks
Jev (TypeSafe AI)	Fast decisions; restrict outputs to choices, scores, and probabilities
Thinking Machines Lab interaction models	Real-time audio/video/text collaboration; responsiveness matters too

All aim to earn a position on the frontier.

Which are non-dominated depends on the **workload and objectives**.

Q1: How to measure the speed of a program?

- **Measure:** wall-clock time, repeated, on stated hardware
- **Software:** instructions $\times$ cycles per instruction $\times$ seconds per cycle
- **Hardware:** parallelism (cores, SIMD, GPUs) and data movement (memory, disk, network)
- Speed is set by the **bottleneck**: copyji is memory-bound
- **Cost** is part of speed: \$ per hour, \$ per GB